

Secure Your PHP Application

In the open source world, PHP programming is always given first preference due to its availability and flexibility. It is a powerful server side scripting language and provides a robust framework to create versatile Web applications. However, a few recent virus attacks on PHP-based websites have shown that this power can be diminished if the programmer doesn't implement a few important security measures. This article talks about important challenges in PHP programming from the security point of view and provides solutions to avoid or mitigate those risks.



Let's quickly look at how PHP works. Typically, PHP runs on an Apache Web server; however, it can also run on commercial Web platforms such as Microsoft IIS. PHP is a set of libraries installed on the Web server side, which forms a framework to process Web requests. It also has the necessary bells and whistles to perform back-end database connectivity and to call Web services. An important part of its engine is to generate dynamic HTML based on the program being executed and communicate it back to the browser over the http connection. PHP does not process HTML, CSS templates or JavaScript. This is because those are client-side scripts and hence are rendered and processed on the browser side.

First, let's talk about website security, in general. No security can be possible without a complete solution around a Web server. For example, a properly configured firewall, hardened operating system, locked down Web server and back-end database are essential for end to end website security. PHP, from the security perspective, is a versatile programming language; however, it doesn't provide any inherent, built-in automatic means to secure code. It is up to the programmer to

understand security problems and ensure that the code handles those situations properly. When we talk about security in a Web application, it boils down to two categories—remote and local. Local exploitation is caused due to the incorrect or imperfect set-up of Web servers and the configuration of the operating system on which it runs. Remote exploitation takes place when malicious users who know how to exploit code level vulnerabilities make their way through the intricate details of the Web deployment system. There are ways to mitigate risks in both these categories. Please study Table 1, which shows the typical security problems with examples.

Table 1

Problem area	Possible exploit
Forms processing	Form submit spoofing
Sessions	Session hijacking
Databases	SQL injection
Shared hosts	File system exposure